COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	10/01/2026	**DATE SUBMITTED:**	10/01/2026
NAME:	Baltazar, Kenji	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. 10.1 **Graphs** **10.1**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE

```cpp
#include <iostream>

//stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

//structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
    //insert new nodes into adjacency list from the given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head; //point the new node to our current head
        return newNode;
        }
        int N; //this is the number of nodes in the graph

    public:
```

```
26          adjNode **head; //adjacency list as array of pointers
27
28          // Constructor
29          DiaGraph(graphEdge edges[], int n, int N) {
30          // allocate new node
31          head = new adjNode*[N]();
32          this->N = N;
33          // initialize head pointer for all vertices
34          for (int i = 0; i < N; ++i)
35              head[i] = nullptr;
36          // construct directed graph by adding edges to it
37          for (unsigned i = 0; i < n; i++) {
38              int start_ver = edges[i].start_ver;
39              int end_ver = edges[i].end_ver;
40              int weight = edges[i].weight;
41              // insert in the beginning
42              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
43              // point head pointer to new node
44              head[start_ver] = newNode;
45          }
46      }
47      // Destructor - Readjusted
48      ~DiaGraph(){
49          for(int i = 0; i < N; i++){
50              delete[] head[i];
```

```
            }
            delete[] head;
        }
    };
    // print all adjacent vertices of given vertex
    void display_AdjList(adjNode* ptr, int i){
        while (ptr != nullptr) {
            std::cout << "(" << i << ", " << ptr->val
                << ", " << ptr->cost << ") ";
            ptr = ptr->next;
        }
        std::cout << std::endl;
    }
    //graph implementation
    int main(){
        // graph edges array.
        graphEdge edges[] = {// (x, y, w) -> edge from x to y with weight w
                             {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
        };
        int N = 6; // Number of vertices in the graph

        // calculate number of edges
        int n = sizeof(edges)/sizeof(edges[0]);
        // construct graph
        DiaGraph diagraph(edges, n, N);
```

```
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;

    for (int i = 0; i < N; i++){
    // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

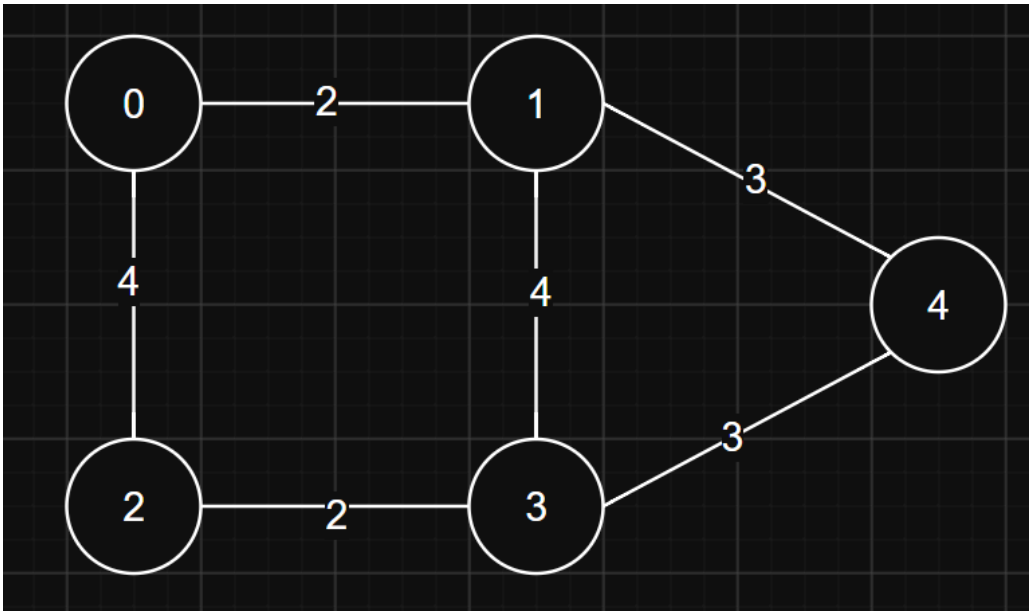
    return 0;
}
```

OUTPUT

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

OBSERVATION

This is the code for graph adjacency where I was able to show the graph just by using the adjacency list. And noticed from the output, we can see the vertices and what vertices are they connected to, we can also see the weight of each path, so we can basically summarize and imagine how the graph would be just from this. In addition to this, I also changed the destructor and added braces because the code keeps on saying that it has an exemption etc.



ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

CODE

```cpp
#ifndef DFS
#define DFS

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
```

```cpp
};
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
          os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph{
    public:
        // Initialize the graph with N vertices
        Graph(size_t N) : V(N){}

        // Return number of vertices in the graph
        auto vertices() const{
            return V;
        }

        // Return all edges in the graph
```

```
51          auto &edges() const{
52              return edge_list;
53          }
54
55          void add_edge(Edge<T> &&e){
56              // Check if the source and destination vertices are within range
57              if (e.src >= 1 && e.src <= V &&
58                  e.dest >= 1 && e.dest <= V)
59                  edge_list.emplace_back(e);
60              else
61                  std::cerr << "Vertex out of bounds" << std::endl;
62          }
63
64          // Returns all outgoing edges from vertex v
65          auto outgoing_edges(size_t v) const{
66              std::vector<Edge<T>> edges_from_v;
67              for (auto &e : edge_list){
68                  if (e.src == v)
69                      edges_from_v.emplace_back(e);
70              }
71              return edges_from_v;
72          }
73
74          template <typename U>
75          friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```
76
77      private:
78          size_t V; // Stores number of vertices in graph
79          std::vector<Edge<T>> edge_list;
80  };
81
82  template <typename T>
83  auto depth_first_search(const Graph<T> &G, size_t dest){
84      std::stack<size_t> stack;
85      std::vector<size_t> visit_order;
86      std::set<size_t> visited;
87      stack.push(1); // Assume that DFS always starts from vertex ID 1
88      while (!stack.empty()){
89          auto current_vertex = stack.top();
90          stack.pop();
91          // If the current vertex hasn't been visited in the past
92          if (visited.find(current_vertex) == visited.end()){
93              visited.insert(current_vertex);
94              visit_order.push_back(current_vertex);
95              for (auto e : G.outgoing_edges(current_vertex)){
96                  // If the vertex hasn't been visited, insert it in the stack.
97                  if(visited.find(e.dest) == visited.end()){
98                      stack.push(e.dest);
99                  }
100             }
```

```
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

#endif
```

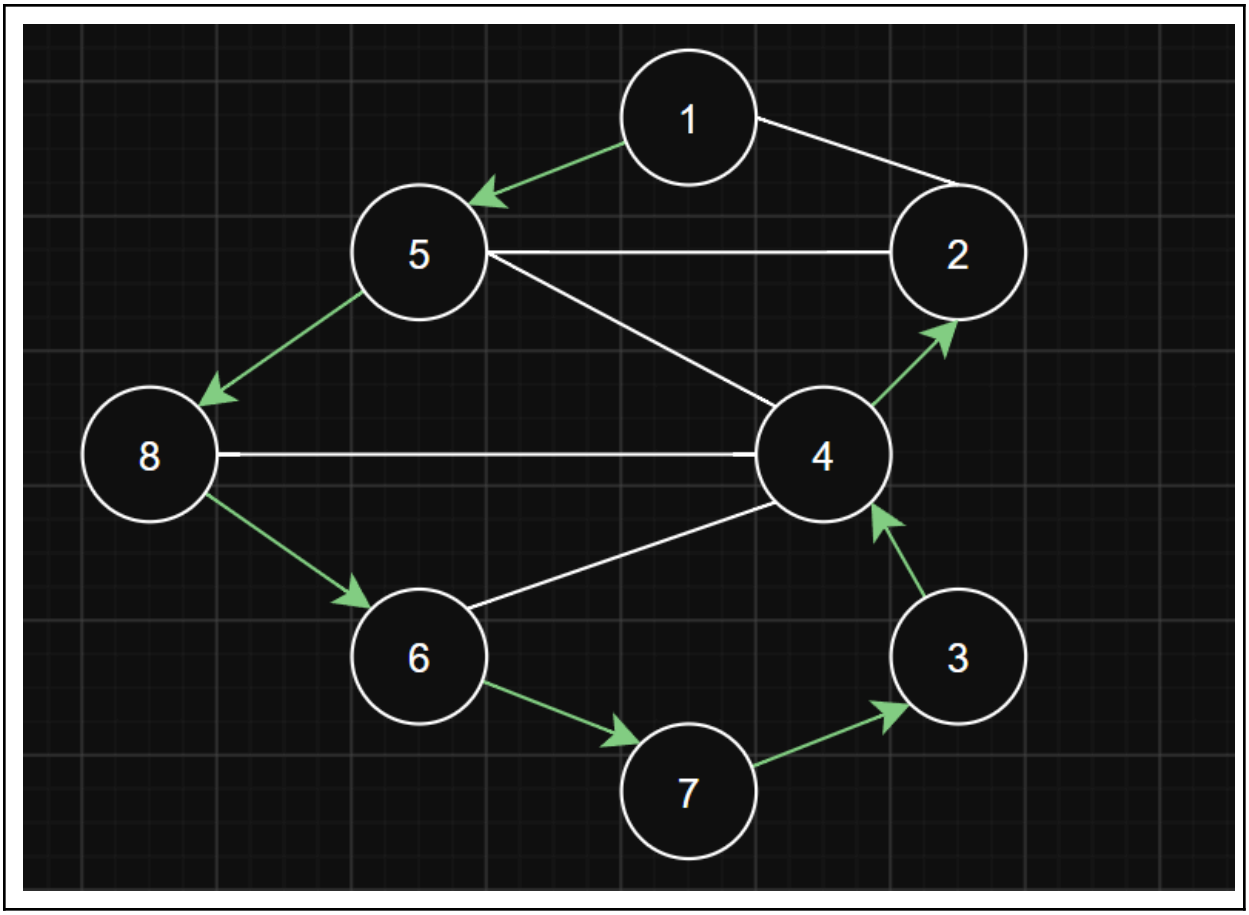
OUTPUT

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

OBSERVATION

This is the code for DFS where the output is 1,5,8,6,7,3,4,2,1 where it first started at vertex 1 then goes down like a pre order traversal would until it reaches the end, where it backtracks to find a vertex with an adjacent vertex that has not been visited yet. The DFS uses stacks.



B.2. Breadth-First Search

CODE

```cpp
#ifndef BFS
#define BFS

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph2;

template <typename T>
struct Edge2{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge2<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge2<T> &e) const{
        return this->weight > e.weight;
    }
```

```
26  };
27
28  template <typename T>
29  std::ostream &operator<<(std::ostream &os, const Graph2<T> &G){
30      for (auto i = 1; i < G.vertices(); i++){
31          os << i << ":\t";
32          auto edges = G.outgoing_edges(i);
33          for (auto &e : edges)
34              os << "{" << e.dest << ": " << e.weight << "}, ";
35          os << std::endl;
36      }
37      return os; //ADDED
38  };
39
40  template <typename T>
41  class Graph2{
42      public:
43          Graph2(size_t N) : V(N){}
44
45          auto vertices() const{
46              return V;
47          }
48
49          auto &edges() const{
50              return edge_list;
```

```
51          }
52
53          void add_edge(Edge2<T> &&e){
54              if (e.src >= 1 && e.src <= V &&
55                  e.dest >= 1 && e.dest <= V)
56                  edge_list.emplace_back(e);
57              else
58                  std::cerr << "Vertex out of bounds" << std::endl;
59          }
60
61          auto outgoing_edges(size_t v) const{
62              std::vector<Edge2<T>> edges_from_v;
63              for (auto &e : edge_list){
64                  if (e.src == v){
65                      edges_from_v.emplace_back(e);
66                  }
67              }
68              return edges_from_v;
69          }
70
71          template <typename>
72          friend std::ostream &operator<<(std::ostream &os, const Graph2<T> &G);
73
74      private:
75          size_t V;
```

```
76          std::vector<Edge2<T>> edge_list;
77  };
78
79  template <typename T>
80  auto create_reference_graph2(){
81      Graph2<T> G(9);
82      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
83      edges[1] = {{2, 2}, {5, 3}};
84      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
85      edges[3] = {{4, 2}, {7, 3}};
86      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
87      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
88      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
89      edges[7] = {{3, 3}, {6, 4}};
90      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
91
92      for (auto &i : edges)
93          for (auto &j : i.second)
94              G.add_edge(Edge2<T>{i.first, j.first, j.second});
95      return G;
96  }
97
98  template <typename T>
99  auto breadth_first_search(const Graph2<T> &G, size_t dest){
100     std::queue<size_t> queue;
```

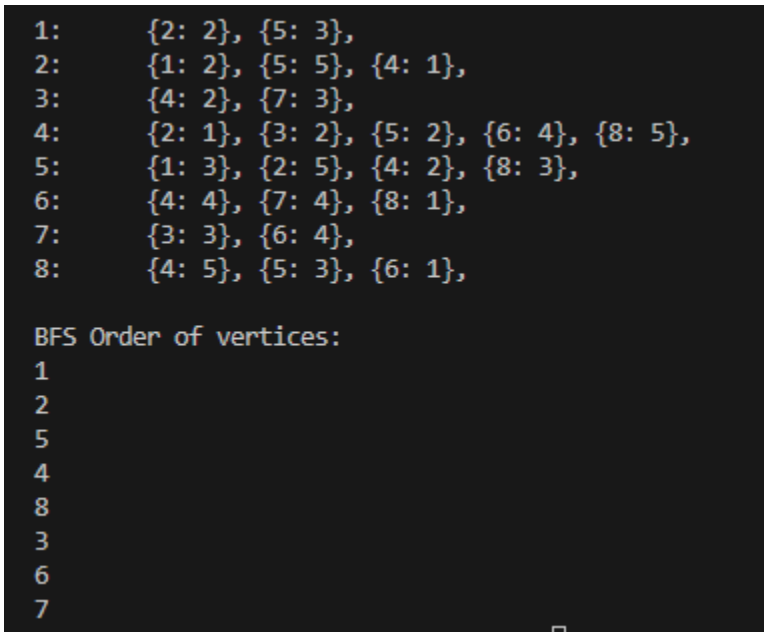
```
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty()){
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph2<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

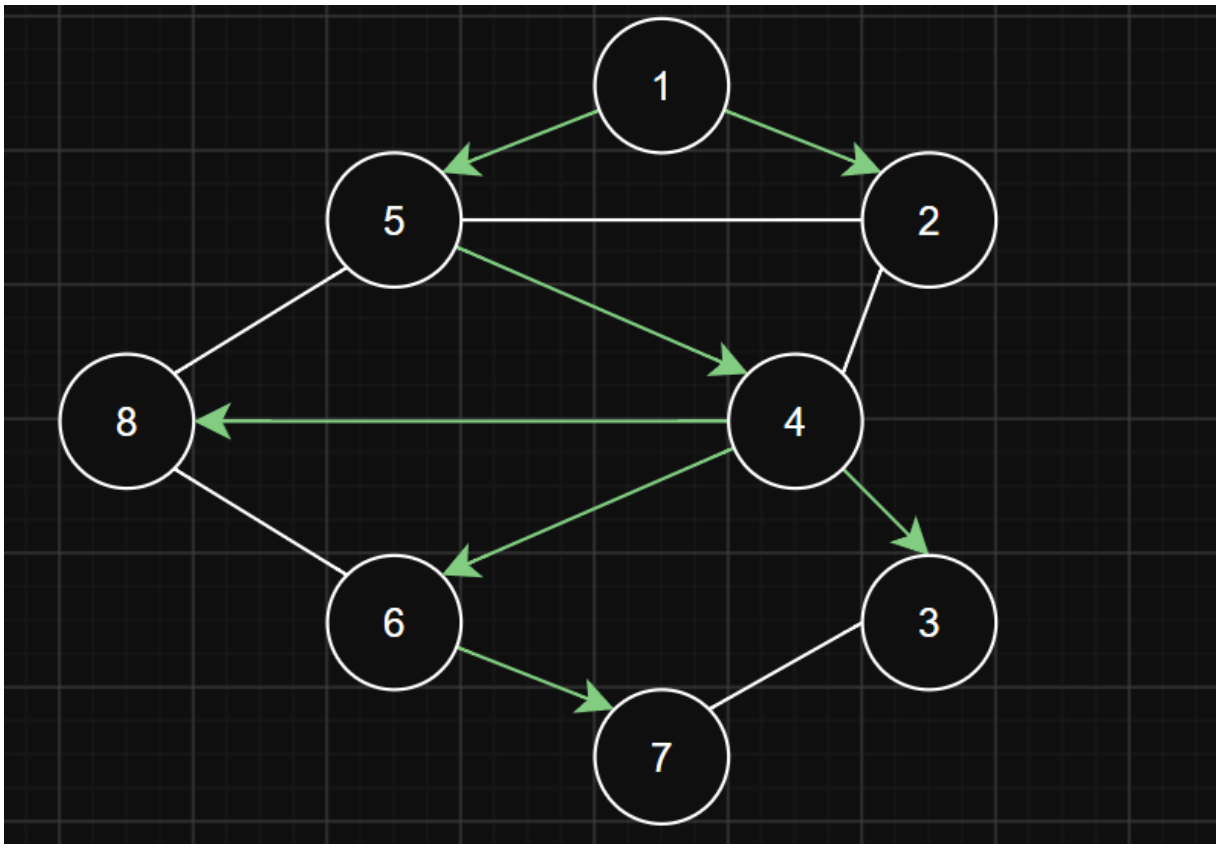
#endif
```

OUTPUT



OBSERVATION

This is the BFS where the output is 1,2,5,4,8,3,6,7 where it follows the level by level rule. So from the starting node it visits adjacent nodes in the same level until all of them are visited, then it goes down one level to further visit more nodes, this will go on until every node has been visited. This uses a queue as well. This specific code has some problems at the start where I need to debug it first to be able to use it again, I tried many times like removing the templating and such but the error is still not fixed until I noticed that there is a difference in this code from the DFS regarding the copy assignment operator, so I just added a return value. Then I noticed next that it seems the name usage for DFS and BFS are the same so I just added 2 to the names for BFS, and I am shocked that it somehow worked. My reasoning for reaching that conclusion is that the error keeps pointing out that the create reference graph is already been initialized or something in that line, so from that point I just thought that why not do it to the other graph and edge as well as there are some errors regarding those.



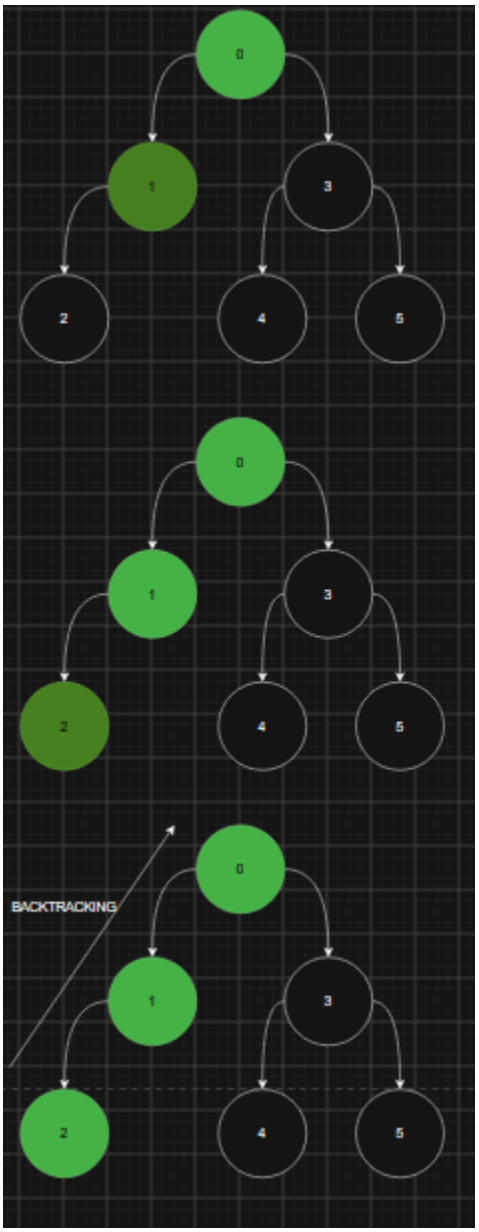
2. ANSWER TO SUPPLEMENTARY ACTIVITY

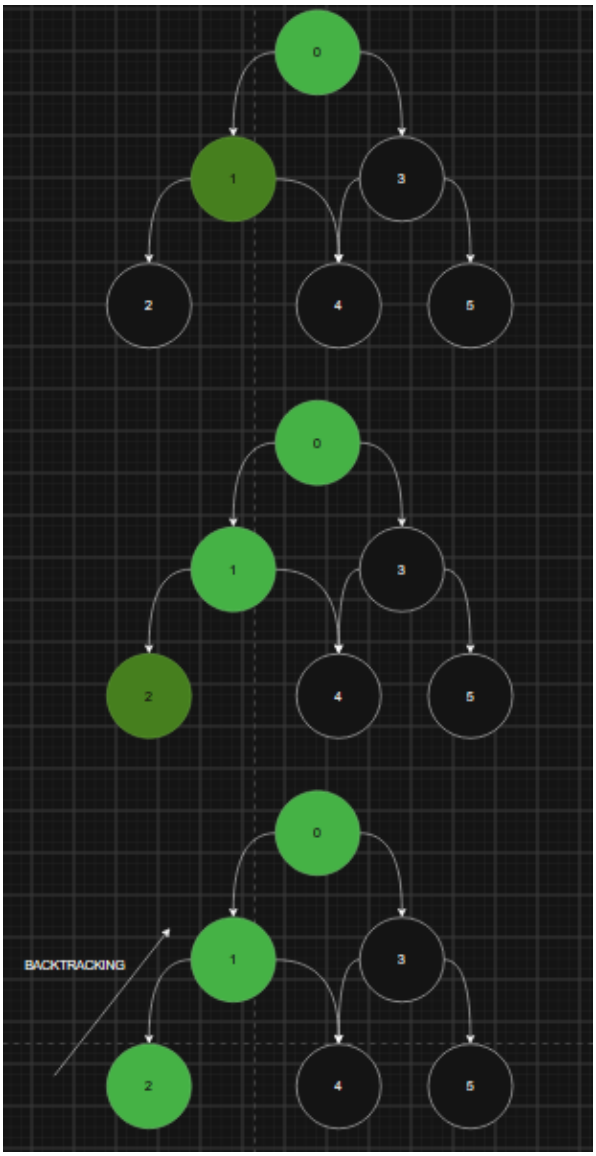
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
   - The most helpful traversal method would be the Depth first search. Because the task mentioned backtracking which is the main point for the DFS, DFS starts from the first vertex then goes down to leftmost then when it reaches the vertex with null at its next it will backtrack and explore other adjacent vertex.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
   - The equivalent for DFS is the preorder traversal. For pre order, we are in the node first then move down to its left child until it reaches the left most leaf then backtracks to go right if there is a right child beside it. This is similar to DFS because both are going to the most left part possible before backtracking. They also have similar pseudocode and code implementation as they both use recursive approach to get deeper to the depths. Although, their main difference is that Pre order is only for trees while DFS is for graphs.

PRE ORDER	DFS





3. In the performed code, what data structure is used to implement the Breadth First Search?
   - For BFS, the data structure used was QUEUE. Because BFS is not backtracking and visits the graph per level, so the vertices that were gonna be discovered first are also visited first, so it needs to follow the first in first out principle.

4. How many times can a node be visited in the BFS?
   - For BFS, nodes can only be visited once because once a node has been visited the algo does not allow backtracking unlike DFS.

3. CONCLUSION

In this hands-on activity, I was able to delve in deeper in learning graphs by implementing the code given in the HOA. I was also able to understand the differences of trees and graphs and was able to apply my knowledge in both in the supplementary activity. First and foremost, I learned that graph is not hierarchical unlike trees and it has 2 types which is directional and undirectional with 2 different cycles as well which is acyclic and cyclic where acyclic means that the vertex in the graph does not go around all edges and goes back to itself while cyclic does it. There are also traversal methods that I learned in this, which are DFS and BFS. DFS or depth first search is just like the pre order search in trees where it checks the left most node and goes deeper until it reaches the last node then backtracks to check the other adjacent nodes. For BFS, I learned that it goes level to level unlike DFS where it first starts at the starting node, goes down one level in the left then checks left to right in that level, then goes down one level until all have been checked, but unlike DFS it does not backtrack and rechecks the already visited nodes. Also, their data structure difference is that BFS uses Queues while DFS uses stacks. Overall, I learned a lot in this HOA and was able to really understand the concepts behind them, I was also able to error check and debug all the codes without any outside help which I am proud of.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.